Markov Decision Processes:
Reinforcement Learning - Lecture 2

Felipe Meneguzzi
felipe.meneguzzi@pucrs.br

PUCRS

Material adapted from:
Sebastian Thrun - Stanford University/Udacity
Sutton and Barto - Reinforcement Learning: An Introduction (2nd ed.)
David Silver - Reinforcement Learning

Outline

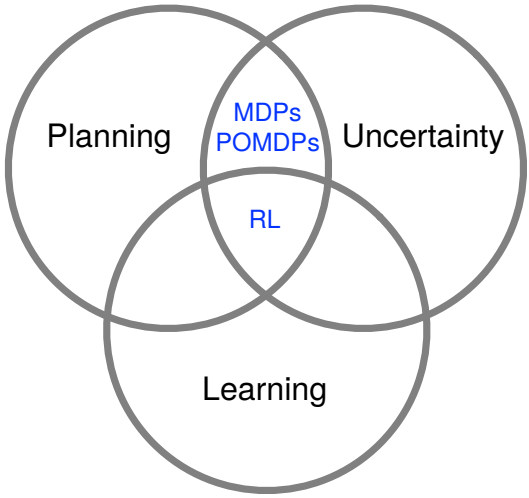
Context

- This lecture (Sutton and Barto 3.1-3.6)
  - Markov Decision Processes
  - Value functions
- Next lecture:
  - **Planning by dynamic programming**
  - Solving a **known** MDP

Recap

- We know how to get from a current state to a goal state
  - Via domain specific search
  - Via domain independent planning
- We also know how to calculate the probability of future events via probability theory
- But, how can we get the agent to decide between different actions, particularly in the face of uncertainty?

Planning Under Uncertainty



Recap

	Deterministic	Stochastic
Fully Observable		
Partially Observable		

Recap

	Deterministic	Stochastic
Fully Observable	A*, DFS BFS, GP	
Partially Observable		

Recap

	Deterministic	Stochastic
Fully Observable	A*, DFS BFS, GP	MDP
Partially Observable		

Recap

	Deterministic	Stochastic
Fully Observable	A*, DFS BFS, GP	MDP
Partially Observable		POMDP

MDPs

- **Markov Decision Processes** (MDPs) formally describe environments for reinforcement learning
- Assumptions behind MDP environments
  - Fully observable states (states completely characterizes the process)
  - Stochastic transitions
  - Agent has **partial control** over transitions
- Almost all RL problems can be formalised as MDPs, e.g.:
  - Optimal control primarily deals with continuous MDPs
  - Partially observable problems can be converted into MDPs
  - Bandits are MDPs with one state

Outline

How Good is an action

- Consider an agent capable of planning and executing different actions to achieve different goals
- Example you can either:
  - Stay home and study
  - Go out and have fun
- Which should you do, and why?

Rationality

- A rational agent should pursue the goal that is, in some sense, "best for it"
- How can we identify whether the achievement of one goal is "better" for the agent than another?

Rationality

- A rational agent should pursue the goal that is, in some sense, "best for it"
- How can we identify whether the achievement of one goal is "better" for the agent than another?
- Utility theory
  - We capture preferences between world states via a **utility function** $U(S)$

$$U : States \rightarrow \mathbb{R}$$

  - In other words, we assign a single number to a state to express the **desirability** of the state

Rationality

- A rational agent should pursue the goal that is, in some sense, "best for it"
- How can we identify whether the achievement of one goal is "better" for the agent than another?
- Utility theory
  - We capture preferences between world states via a **utility function** $U(S)$

$$U : States \rightarrow \mathbb{R}$$

  - In other words, we assign a single number to a state to express the **desirability** of the state
    Winning the lottery $U(w) = 1000$

Rationality

- A rational agent should pursue the goal that is, in some sense, "best for it"
- How can we identify whether the achievement of one goal is "better" for the agent than another?
- Utility theory
  - We capture preferences between world states via a **utility function** $U(S)$

$$U : States \rightarrow \mathbb{R}$$

  - In other words, we assign a single number to a state to express the **desirability** of the state
    Winning the lottery $U(w) = 1000$ Failing AI: $U(f) = -10$

Rationality

- A rational agent should pursue the goal that is, in some sense, "best for it"
- How can we identify whether the achievement of one goal is "better" for the agent than another?
- Utility theory
  - We capture preferences between world states via a **utility function** $U(S)$

$$U : States \rightarrow \mathbb{R}$$

  - In other words, we assign a single number to a state to express the **desirability** of the state
    Winning the lottery $U(w) = 1000$ Failing AI: $U(f) = -10$
  - But what if actions have uncertain effects?
    Buying a lottery ticket does not give us a utility of 1000

Expected Utility

- A nondeterministic action A could lead to several possible outcomes, e.g. $Result_1(A), Result_2(A), \ldots$
  $Result(BuyTicket) \in \{win, \neg win\}$
- We can associate a probability with the different outcomes

$$\mathbb{P}\left[Result_i(A) | Do(A), E\right]$$

  - E captures the evidence we have about the world
  - $Do(A)$ is the proposition that action A is executed in the current state
- The **expected utility** of executing A given E is then

$$\mathbb{E}\left[U(A) \mid E\right] = \sum_i \mathbb{P}\left[Result_i(A) \mid Do(A), E\right] U(Result_i(A))$$

- Principle of **maximum expected utility** (MEU):
  A rational agent should choose the action that maximises its expected utility

More generally

- We can use the expectation operator ($\mathbb{E}[X]$) for **any random variable** X

$$\mathbb{E}[X] \doteq \sum_{x \in X} \mathbb{P}[x]\, x$$

- Example:
  Expected value of a six sided die:

More generally

- We can use the expectation operator ($\mathbb{E}[X]$) for **any random variable** X

$$\mathbb{E}[X] \doteq \sum_{x \in X} \mathbb{P}[x]\, x$$

- Example:
  Expected value of a six sided die:

x	P(X=x)	x*P(X=x)
1	1/6	1/6
2	1/6	2/6
3	1/6	3/6
4	1/6	4/6
5	1/6	5/6
6	1/6	6/6
	Total	21/6 = 3.5

Maximum Expected Utility

- An agent following the principle of MEU is rational, acting optimally in any situation
- But choosing the best sequence of actions requires enumerating all action sequences; infeasible for long sequences
- Computations are often too expensive:
  - $\mathbb{P}[Result_i(A)|Do(A), E]$ requires a complete causal model of the world. Even if all information is available, this is computationally intractable
  - Computing $U(Result_i(A))$ requires searching or planning as we do not know how good a state is until we know what goals we can reach from it (and how expensive they are)

Maximum Expected Utility II

- MEU is no silver bullet for the problems of AI
  - But does give us a nice framework for building an agent,
  - An MEU maximising agent will achieve the highest possible performance averaged over all environments in which the agent can be placed
  - This moves from a performance measure in the external environment, to an internal criteria of utility maximisation

Lotteries

- A lottery is a probability distribution over a set of actual outcomes
- A lottery L with possible outcomes $C_1, \ldots, C_n$ and associated probabilities $p_1, \ldots, p_n$ is written as

$$L = [p_1, C_1; \ldots p_n, C_n]$$

- Each outcome C_i is an atomic state, **or another lottery**

Lotteries

- A lottery is a probability distribution over a set of actual outcomes
- A lottery L with possible outcomes $C_1, \ldots, C_n$ and associated probabilities $p_1, \ldots, p_n$ is written as
$$L = [p_1, C_1; \ldots p_n, C_n]$$
- Each outcome C_i is an atomic state, **or another lottery**
- Example: There is a 30% likelihood I will go out tonight.
If I do so, there is a 60% likelihood I will do badly in my test.
If I do not go out, I will do well in my test with a 90% likelihood

$$L = [0.3, [0.6, f; 0.4, p]; 0.7, [0.9, p; 0.1, f]]$$

Lotteries

- A lottery is a probability distribution over a set of actual outcomes
- A lottery L with possible outcomes $C_1, \ldots, C_n$ and associated probabilities $p_1, \ldots, p_n$ is written as
$$L = [p_1, C_1; \ldots p_n, C_n]$$
- Each outcome C_i is an atomic state, **or another lottery**
- Example: There is a 30% likelihood I will go out tonight.
If I do so, there is a 60% likelihood I will do badly in my test.
If I do not go out, I will do well in my test with a 90% likelihood
$$L = [0.3, [0.6, f; 0.4, p]; 0.7, [0.9, p; 0.1, f]]$$
- Our goal is to understand the relationships between preferences between lotteries and preferences between states

- **Maximum Expected Utility Principle**: The utility of a lottery is the sum of the probability of each outcome times the utility of that outcome

$$U([p_1, S_1; \ldots, p_n, S_n]) = \sum_i p_i U(S_i)$$

- **Maximum Expected Utility Principle**: The utility of a lottery is the sum of the probability of each outcome times the utility of that outcome

$$U([p_1, S_1; \ldots, p_n, S_n]) = \sum_i p_i U(S_i)$$

- Given probabilities and utilities of outcomes, we can determine the utility of a compound lottery

- **Maximum Expected Utility Principle**: The utility of a lottery is the sum of the probability of each outcome times the utility of that outcome

$$U([p_1, S_1; \ldots, p_n, S_n]) = \sum_i p_i U(S_i)$$

- Given probabilities and utilities of outcomes, we can determine the utility of a compound lottery
- Since outcomes of nondeterministic actions are lotteries, we obtain the MEU decision rule

- **Maximum Expected Utility Principle**: The utility of a lottery is the sum of the probability of each outcome times the utility of that outcome

$$U([p_1, S_1; \ldots, p_n, S_n]) = \sum_i p_i U(S_i)$$

- Given probabilities and utilities of outcomes, we can determine the utility of a compound lottery
- Since outcomes of nondeterministic actions are lotteries, we obtain the MEU decision rule
  - We now know what a utility function is
  - But without structure, there is little we can do with it

Utility Functions

- A typical use of utility functions (stemming from economic theory) maps the amount of money an agent has to its utility
- Agents prefer having more money to less: such a preference is **monotonic**
- But what about lotteries about money?

Utility Functions

- A typical use of utility functions (stemming from economic theory) maps the amount of money an agent has to its utility
- Agents prefer having more money to less: such a preference is **monotonic**
- But what about lotteries about money?
- You are a participant in a game show. You are in the final round, and can take \$1,000,000, or gamble it on a coin flip.
  If the coin lands on heads, you will win \$3,000,000, tails, you will take home nothing.
  What do you do?

Utility Functions

- A typical use of utility functions (stemming from economic theory) maps the amount of money an agent has to its utility
- Agents prefer having more money to less: such a preference is **monotonic**
- But what about lotteries about money?
- You are a participant in a game show. You are in the final round, and can take \$1,000,000, or gamble it on a coin flip.
  If the coin lands on heads, you will win \$3,000,000, tails, you will take home nothing.
  What do you do?
- The **expected monetary value** (EMV) is \$1,500,000;
  this is more than the prize
- If \$1=1 utility, then a rational agent would take the gamble

- Instead, let us denote the state of having $\$n$ as S_n, and the your initial states as S_k. Then

$$EU(flip) = 0.5U(S_k) + 0.5U(S_{k+3000000}) \qquad EU(noflip) = U(S_{k+1000000})$$

- We now need to assign utilities to the different outcome states
  - Utility is typically not proportional to money:
    having a 3,000,000 is **not** worth 3 times as much as having 1,000,000.

Utility Functions

- Instead, let us denote the state of having $\$n$ as S_n, and the your initial states as S_k. Then

$$EU(flip) = 0.5U(S_k) + 0.5U(S_{k+3000000}) \qquad EU(noflip) = U(S_{k+1000000})$$

- We now need to assign utilities to the different outcome states
  - Utility is typically not proportional to money:
    having a 3,000,000 is **not** worth 3 times as much as having 1,000,000.
  - So perhaps $S_k = 5, S_{k+3000000} = 10, S_{k+1000000} = 8$

- Instead, let us denote the state of having $\$n$ as S_n, and the your initial states as S_k. Then

$$EU(flip) = 0.5U(S_k) + 0.5U(S_{k+3000000}) \qquad EU(noflip) = U(S_{k+1000000})$$

- We now need to assign utilities to the different outcome states
  - Utility is typically not proportional to money:
    having a 3,000,000 is **not** worth 3 times as much as having 1,000,000.
  - So perhaps $S_k = 5, S_{k+3000000} = 10, S_{k+1000000} = 8$
  - What are the expected utilities? $EU(flip) =?$ $EU(noflip) =?$

- Instead, let us denote the state of having $\$n$ as S_n, and the your initial states as S_k. Then

$$EU(flip) = 0.5U(S_k) + 0.5U(S_{k+3000000}) \qquad EU(noflip) = U(S_{k+1000000})$$

- We now need to assign utilities to the different outcome states
  - Utility is typically not proportional to money:
    having a 3,000,000 is **not** worth 3 times as much as having 1,000,000.
  - So perhaps $S_k = 5, S_{k+3000000} = 10, S_{k+1000000} = 8$
  - Then $EU(flip) = 7.5, EU(noflip) = 8$, and you would not flip the coin

- Instead, let us denote the state of having $\$n$ as S_n, and the your initial states as S_k. Then

$$EU(flip) = 0.5U(S_k) + 0.5U(S_{k+3000000}) \qquad EU(noflip) = U(S_{k+1000000})$$

- We now need to assign utilities to the different outcome states
  - Utility is typically not proportional to money:
    having a 3,000,000 is **not** worth 3 times as much as having 1,000,000.
  - So perhaps $S_k = 5, S_{k+3000000} = 10, S_{k+1000000} = 8$
  - Then $EU(flip) = 7.5, EU(noflip) = 8$, and you would not flip the coin
- Typically, for some lottery L, $U(L) < U(S_{EMV(L)})$. In other words, being given the cash without gambling is better than facing a lottery
  - Agents with this type of utility function are **risk averse**
  - If $U(S_{EMV(L)}) < U(L)$ the agent is **risk seeking**

Utility Functions

- The amount an agent will accept instead of a lottery is called the **certainty equivalent** of the lottery
- The difference between the expected monetary value of a lottery and its certainty equivalent is called the **insurance premium**
- The insurance premium is only positive if an agent is risk averse
- If a utility curve is linear (i.e. $U(S_{EMV(L)}) = U(L)$), the agent is

Utility Functions

- The amount an agent will accept instead of a lottery is called the **certainty equivalent** of the lottery
- The difference between the expected monetary value of a lottery and its certainty equivalent is called the **insurance premium**
- The insurance premium is only positive if an agent is risk averse
- If a utility curve is linear (i.e. $U(S_{EMV(L)}) = U(L)$), the agent is **risk neutral**
- Any affine transformation of a utility function is a utility function

Outline

Markov Property

The future is independent of the past, given the present

Definition

A state S_t is **Markov** if and only if

$$\mathbb{P}\left[S_{t+1} \mid S_t\right] = \mathbb{P}\left[S_{t+1} \mid S_1, \ldots S_t\right]$$

- The state captures all relevant information from the history
- Once the state is known, the history may be thrown away
  i.e. The state is a sufficient statistic of the future

State Transition Matrix

For a Markov state s and successor state s', the **state transition probability** is defined by

$$\mathcal{P}_{ss'} = \mathbb{P}\left[S_{t+1} = s' \mid S_t = s\right]$$

State transition matrix $\mathcal{P}$ defines transition probabilities from all states s to all successor states s',

$$\mathcal{P} = \begin{array}{c} \\ \textit{from} \end{array} \begin{array}{c} \begin{array}{ccc} & \textit{to} & \end{array} \\ \begin{bmatrix} \mathcal{P}_{11} & \dots & \mathcal{P}_{1n} \\ \vdots & & \\ \mathcal{P}_{n1} & \dots & \mathcal{P}_{nn} \end{bmatrix} \end{array}$$

where each row of the matrix sums to 1.

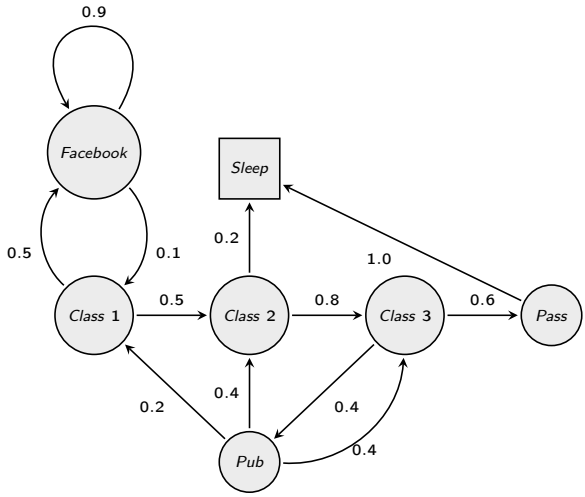
A Markov process is a memoryless random process, i.e. a sequence of random states S_1, S_2, ... with the Markov property.

Definition

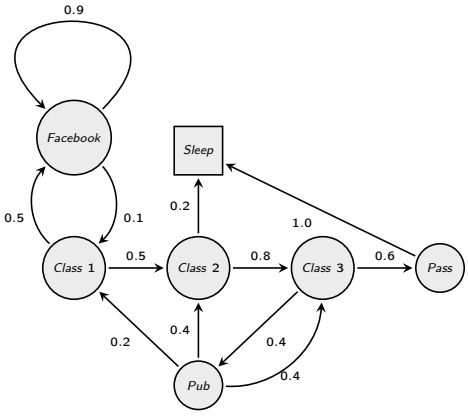
A Markov Process (or **Markov Chain**) is a tuple $\langle \mathcal{S}, \mathcal{P} \rangle$

- $\mathcal{S}$ is a (finite) set of states
- $\mathcal{P}$ is a state transition probability matrix,
  $\mathcal{P}_{ss'} = \mathbb{P}[S_{t+1} = s' \mid S_t = s]$

Example: Student Markov Chain



Example: Student Markov Chain Episodes

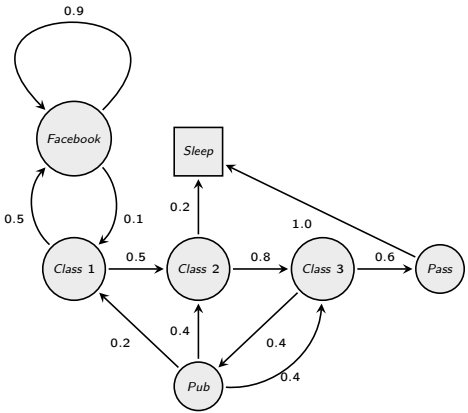


Sample **episodes** for Student Markov Chain starting from $S_1 = C1$

$$S_1, S_2, \ldots, S_T$$

- C1 C2 C3 Pass Sleep
- C1 FB FB C1 C2 Sleep
- C1 C2 C3 Pub C2 C3 Pass Sleep
- C1 FB FB C1 C2 C3 Pub C1 FB FB FB C1 C2 C3 Pub C2 Sleep

Example: Student Markov Chain Transition Matrix



$$\mathcal{P} = \begin{array}{c} \\ C1 \\ C2 \\ C3 \\ Pass \\ Pub \\ FB \\ Sleep \end{array} \begin{array}{c} \begin{array}{ccccccc} C1 & C2 & C3 & Pass & Pub & FB & Sleep \end{array} \\ \begin{bmatrix} & 0.5 & & & & 0.5 & \\ & & 0.8 & & & & 0.2 \\ & & & 0.6 & 0.4 & & \\ & & & & & & 1.0 \\ 0.2 & 0.4 & 0.4 & & & & \\ 0.1 & & & & & 0.9 & \\ & & & & & & 1.0 \end{bmatrix} \end{array}$$

Outline

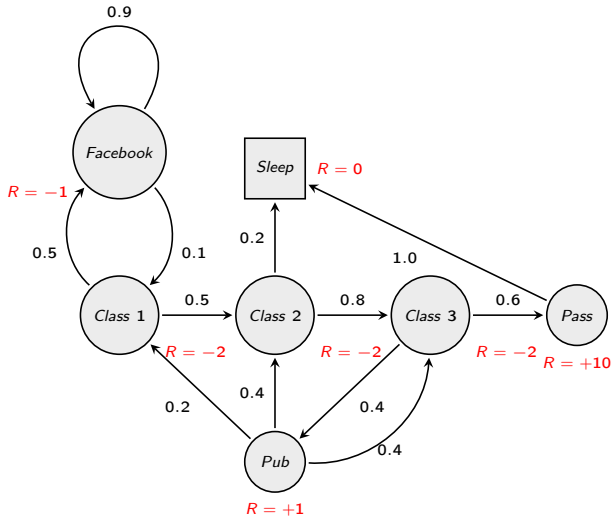
A Markov Reward Processes is a Markov chain with values

Definition

A **Markov Reward Process** is a tuple $\langle \mathcal{S}, \mathcal{P}, \mathcal{R}, \gamma \rangle$

- $\mathcal{S}$ is a finite set of states
- $\mathcal{P}$ is a transition probability matrix/function, $\mathcal{P}_{ss'} = \mathbb{P}[S_{t+1} = s' \mid S_t = s]$
- $\mathcal{R}$ is a reward function, $R_s = \mathbb{E}[R_{t+1} \mid S_t = s]$
- γ is a discount factor, $\gamma \in [0, 1]$

Example: Student MRP



Definition

The return G_t is the total discounted reward from time-step t.

$$G_t = R_{t+1} + \gamma R_{t+2} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

- The discount $\gamma \in [0, 1]$ is the present value of the future rewards
- The value of receiving reward R after $k + 1$ time steps is $\gamma^k R$
- This values immediate reward above delayed reward
  - γ close to 0 leads to "myopic" evaluation
  - γ close to 1 leads to "far-sighted" evaluation

Why discount?

Most Markov reward and decision processes are discounted. Why?

- Mathematically convenient to discount rewards
- Avoids infinite returns in cyclic Markov processes
- Uncertainty about the future may not be fully represented
- If the reward is financial, immediate rewards may earn more interest than delayed rewards
- Animal/human behaviour shows preference for immediate reward
- It is sometimes possible to use undiscounted Markov reward processes (i.e. $\gamma = 1$), e.g. if all sequences terminate.

Value function

The value function $v(s)$ gives the long-term value of state s

Definition

The **state value function** $v(s)$ of an MRP is the expected return starting from state s

$$v(s) = \mathbb{E}\left[G_t \mid S_t = s\right]$$

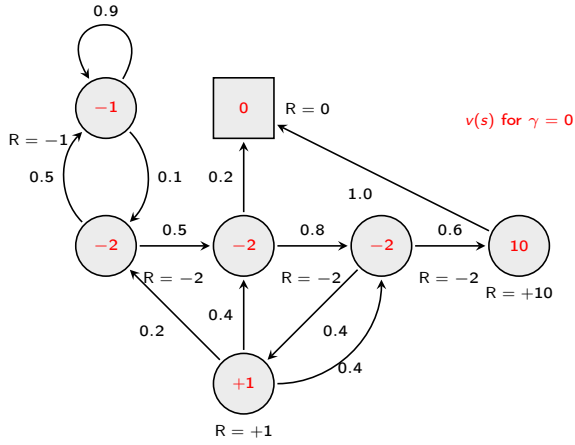
Example: Student MRP Returns

Sample **returns** for Student MRP: Starting from $S_1 = C1$ with $\gamma = \frac{1}{2}$

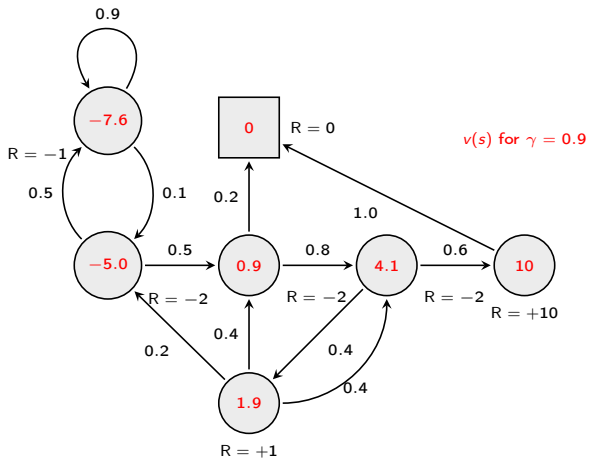
$$G_1 = R_2 + \gamma R_3 + \cdots + \gamma^{T-2} R_T$$

C1 C2 C3 Pass Sleep	$v_1 = -2 - 2 * \frac{1}{2} - 2 * \frac{1}{4} + 10 * \frac{1}{8} = -2.25$
C1 FB FB C1 C2 Sleep	$v_1 = -2 - 1 * \frac{1}{2} - 1 * \frac{1}{4} + 2 * \frac{1}{8} - 2 * \frac{1}{16} = -3.125$
C1 C2 C3 Pub C2 C3 Pass Sleep	$v_1 = -2 - 2 * \frac{1}{2} - 2 * \frac{1}{4} + 1 * \frac{1}{8} - 2 * \frac{1}{16} = -3.41$
C1 FB FB C1 C2 C3 Pub C1 ...	$v_1 = -2 - 1 * \frac{1}{2} - 1 * \frac{1}{4} - 2 * \frac{1}{8} - 2 * \frac{1}{16} = -3.20$
FB FB FB C1 C2 C3 Pub C2 Sleep	

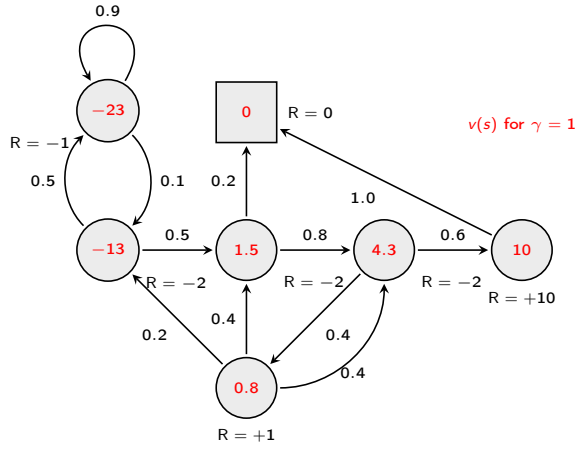
Example: State-Value Function for Student (MRP) (1)



Example: State-Value Function for Student (MRP) (2)



Example: State-Value Function for Student (MRP) (3)



Bellman Equation for MRPs

The value function can be decomposed into two parts:

- immediate reward R_{t+1}
- discounted value of successor state $\gamma\, v(S_{t+1})$

$$v(s) = \mathbb{E}\left[G_t \mid S_t = s\right]$$

Bellman Equation for MRPs

The value function can be decomposed into two parts:

- immediate reward R_{t+1}
- discounted value of successor state $\gamma\, v(S_{t+1})$

$$
\begin{aligned}
v(s) &= \mathbb{E}\left[G_t \mid S_t = s\right] \\
&= \mathbb{E}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \mid S_t = s\right]
\end{aligned}
$$

Bellman Equation for MRPs

The value function can be decomposed into two parts:

- immediate reward R_{t+1}
- discounted value of successor state $\gamma\, v(S_{t+1})$

$$
\begin{aligned}
v(s) &= \mathbb{E}\left[G_t \mid S_t = s\right] \\
&= \mathbb{E}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \mid S_t = s\right] \\
&= \mathbb{E}\left[R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \dots) \mid S_t = s\right]
\end{aligned}
$$

Bellman Equation for MRPs

The value function can be decomposed into two parts:

- immediate reward R_{t+1}
- discounted value of successor state $\gamma\, v(S_{t+1})$

$$
\begin{aligned}
v(s) &= \mathbb{E}\left[G_t \mid S_t = s\right] \\
&= \mathbb{E}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \mid S_t = s\right] \\
&= \mathbb{E}\left[R_{t+1} + \gamma (R_{t+2} + \gamma R_{t+3} + \dots) \mid S_t = s\right] \\
&= \mathbb{E}\left[R_{t+1} + \gamma\, G_{t+1} \mid S_t = s\right]
\end{aligned}
$$

Bellman Equation for MRPs

The value function can be decomposed into two parts:

- immediate reward R_{t+1}
- discounted value of successor state $\gamma\, v(S_{t+1})$

$$
\begin{aligned}
v(s) &= \mathbb{E}\left[G_t \mid S_t = s\right] \\
&= \mathbb{E}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \mid S_t = s\right] \\
&= \mathbb{E}\left[R_{t+1} + \gamma (R_{t+2} + \gamma R_{t+3} + \ldots) \mid S_t = s\right] \\
&= \mathbb{E}\left[R_{t+1} + \gamma G_{t+1} \mid S_t = s\right] \\
&= \mathbb{E}\left[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s\right]
\end{aligned}
$$

Bellman Equation for MRPs (2)

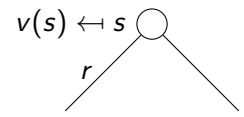
$$v(s) = \mathbb{E}\left[? \mid S_t = s\right]$$

$$v(s) \leftarrow\!\shortmid\ s \bigcirc$$

$$v(s) = ?$$

Bellman Equation for MRPs (2)

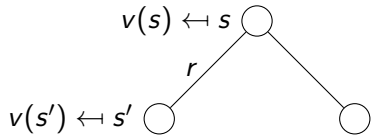
$$v(s) = \mathbb{E}\left[\mathrm{R}_{t+1} \mid \mathrm{S}_t = s\right]$$



$$v(s) = \mathcal{R}_s$$

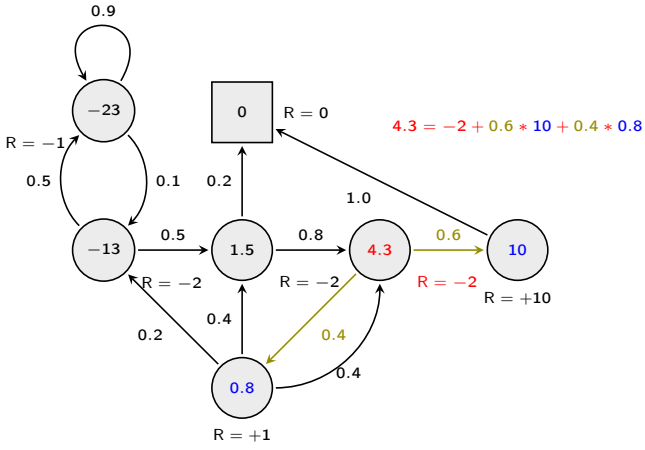
Bellman Equation for MRPs (2)

$$v(s) = \mathbb{E}\left[R_{t+1} + \gamma\, v(S_{t+1}) \mid S_t = s\right]$$



$$v(s) = \mathcal{R}_s + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}\, v(s')$$

Example: Bellman Equation for Student MRP



Bellman Equation in Matrix Form

The Bellman equation can be expressed concisely using matrices,

$$v = \mathcal{R} + \gamma \mathcal{P}\, v$$

where v is a column vector with one entry per state

$$\begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix} = \begin{bmatrix} \mathcal{R}_1 \\ \vdots \\ \mathcal{R}_n \end{bmatrix} + \gamma \begin{bmatrix} \mathcal{P}_{11} & \dots & \mathcal{P}_{1n} \\ \vdots & & \\ \mathcal{P}_{n1} & \dots & \mathcal{P}_{nn} \end{bmatrix} \begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix}$$

Solving the Bellman Equation

- The Bellman equation is a linear equation
- It can be solved directly:

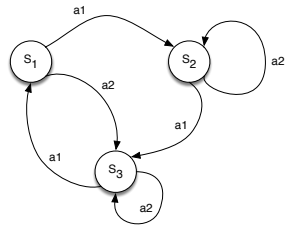
$$
\begin{aligned}
v &= \mathcal{R} + \gamma \mathcal{P} v \\
(\mathrm{I} - \gamma \mathcal{P}) v &= \mathcal{R} \\
v &= (\mathrm{I} - \gamma \mathcal{P})^{-1} \mathcal{R}
\end{aligned}
$$

- Computational complexity is $O(n^3)$ for n states
- Direct solution only possible for small MRPs
- There are many iterative methods for large MRPs, e.g.
  - Dynamic programming
  - Monte-Carlo evaluation
  - Temporal-Difference learning

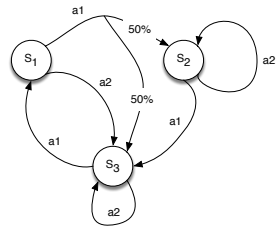
Outline

Markov Decision Process (MDP)

- Such a model, in a fully observable environment, is called a **Markov Decision Process**

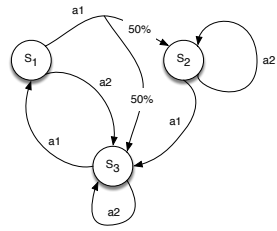


Markov Decision Process (MDP)



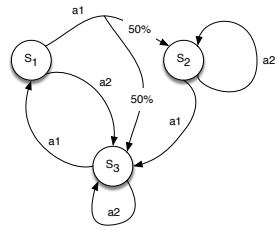
- Such a model, in a fully observable environment, is called a **Markov Decision Process**
- An MDP is defined in terms of
  1. An initial state S_0
  2. A transition model $T(s, a, s')$
  3. A reward function $R(s)$

Markov Decision Process (MDP)



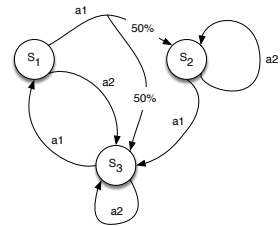
- Such a model, in a fully observable environment, is called a **Markov Decision Process**
- An MDP is defined in terms of
  1. An initial state S_0
  2. A transition model $T(s, a, s') \text{ — } P(s'|a, s)$ (**Markovian**)
  3. A reward function $R(s)$ — sometimes expressed as $R(a, s)$

Markov Decision Process (MDP)



- Such a model, in a fully observable environment, is called a **Markov Decision Process**
- An MDP is defined in terms of
  1. An initial state S_0
  2. A transition model $T(s, a, s') \text{ — } P(s'|a, s)$ (**Markovian**)
  3. A reward function $R(s)$ — sometimes expressed as $R(a, s)$
- A solution to a MDP must specify what the agent should do for any state. Such a solution is called a **policy**

Markov Decision Process (MDP) - In Sutton and Barto's Book



- Such a model, in a fully observable environment, is called a **Markov Decision Process**
- An MDP is defined in terms of:
  1. A finite set of states $\mathcal{S}$
  2. A finite set of actions $\mathcal{A}$
  3. A markovian transition model $T(s, a, s') = \mathbb{P}(S_{t+1} = s' \mid S_t = s, A_t = a)$
  4. A reward function $R(s) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$
  5. A discount factor $\gamma \in [0, 1]$
- A solution to a MDP must specify what the agent should do for any state. Such a solution is called a **policy**

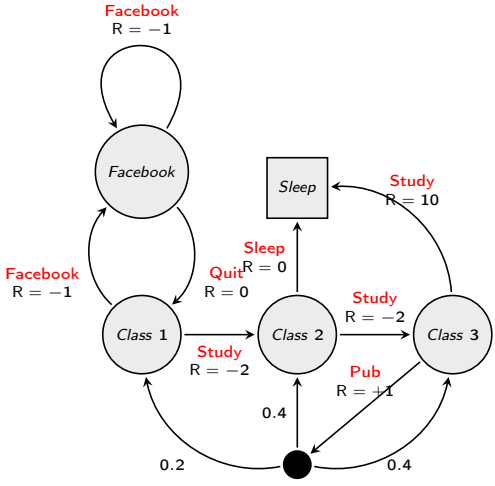
A Markov decision process (MDP) is a Markov reward process with decisions. It is an environment in which all states are Markov.

Definition

A Markov Decision Process is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$:

1. A finite set of states $\mathcal{S}$
2. A finite set of actions $\mathcal{A}$
3. A markovian transition model $\mathcal{P}^{a}_{ss'} = \mathbb{P}(S_{t+1} = s' \mid S_t = s, A_t = a)$
4. A reward function $\mathcal{R}^{a}_{s} = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$
5. A discount factor $\gamma \in [0, 1]$

Student MDP Example



Deterministic Policies

- We denote a policy by π
- $\pi(s)$ identifies the action recommended by the policy in state s
- A complete policy tells an agent what to do no matter the outcome of an action
  - It explicitly represents the agent function and therefore describes a simple reflex agent, computed from the information used for a utility based agent

Deterministic Policies

- We denote a policy by π
- $\pi(s)$ identifies the action recommended by the policy in state s
- A complete policy tells an agent what to do no matter the outcome of an action
  - It explicitly represents the agent function and therefore describes a simple reflex agent, computed from the information used for a utility based agent
  - The quality of a policy is measured by the **expected** utility of the possible environment histories generated by executing that policy

Deterministic Policies

- We denote a policy by π
- $\pi(s)$ identifies the action recommended by the policy in state s
- A complete policy tells an agent what to do no matter the outcome of an action
  - It explicitly represents the agent function and therefore describes a simple reflex agent, computed from the information used for a utility based agent
  - The quality of a policy is measured by the **expected** utility of the possible environment histories generated by executing that policy
  - An **optimal policy** is one that yields the highest expected utility, and is denoted π^*

Stochastic Policies (1)

Definition

A **stochastic** policy π is a distribution over actions given states,

$$\pi(a \mid s) = \mathbb{P}\left[A_t = a \mid S_t = s\right]$$

- A policy fully defines the behaviour of an agent
- MDP policies generally depend on the current state (not the history)
- i.e. Policies are stationary (time-independent),
  $A_t \sim \pi(\cdot \mid S_t), \forall t > 0$

Stochastic Policies (2)

- Given an MDP $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ and a policy π
- The state sequence $S_1, S_2, \ldots$ is a Markov Process $\langle \mathcal{S}, \mathcal{P}^\pi \rangle$
- The state and reward sequence $S_1, R_2, S_2, \ldots$ is a Markov Reward Process $\langle \mathcal{S}, \mathcal{P}^\pi, \mathcal{R}^\pi, \gamma \rangle$, where

$$\mathcal{P}^\pi_{ss'} = \sum_{a \in \mathcal{S}} \pi(a \mid s)\, \mathcal{P}^a_{ss'}$$

$$\mathcal{R}^\pi_s = \sum_{a \in \mathcal{S}} \pi(a \mid s)\, \mathcal{R}^a_s$$

Value Functions (Sutton and Barto)

Definition

The **state-value function** $v_\pi(s)$ of an MDP is the expected return starting from state s, and then following policy π

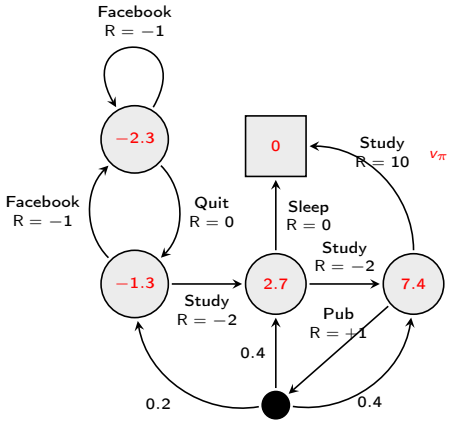
$$v_\pi(s) = \mathbb{E}_\pi \left[G_t \mid S_t = s \right]$$

Definition

The **action-value function** $q_\pi(s, a)$ is the expected return starting from state s, taking action a and then following policy π

$$q_\pi(s, a) = \mathbb{E}_\pi \left[G_t \mid S_t = s, A_t = a \right]$$

Example: State-Value Function for Student MDP



Bellman Expectation Equation

The state-value function can again be decomposed into immediate reward plus discounted value of successor state,

$$v_\pi(s) = \mathbb{E}_\pi \left[G_t \mid S_t = s \right]$$

The action-value function can similarly be decomposed,

$$q_\pi(s, a) = \mathbb{E}_\pi \left[G_t \mid S_t = s, A_t = a \right]$$

Bellman Expectation Equation

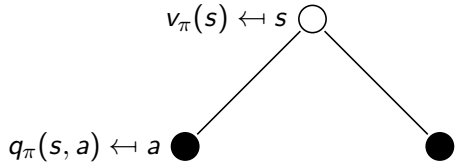
The state-value function can again be decomposed into immediate reward plus discounted value of successor state,

$$v_\pi(s) = \mathbb{E}_\pi \left[R_{t+1} + \gamma \, v_\pi(S_{t+1}) \mid S_t = s \right]$$

The action-value function can similarly be decomposed,

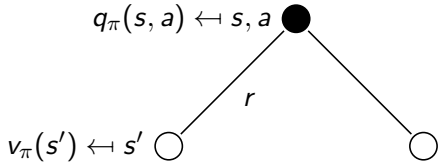
$$q_\pi(s, a) = \mathbb{E}_\pi \left[R_{t+1} + \gamma \, q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a \right]$$

Bellman Expectation Equation for v_π



$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) q_\pi(s, a)$$

Bellman Expectation Equation for Q_π



$$q_\pi(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \, v_\pi(s')$$

Bellman Expectation Equation for v_π (2)

For deterministic policies $\pi(s)$ and state-based rewards $\mathcal{R}_s$

$$v_\pi(s) \leftarrow$$

Bellman Expectation Equation for v_π (2)

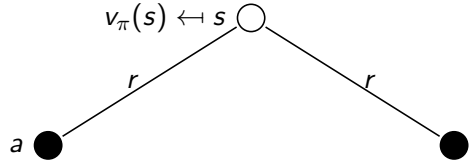
For deterministic policies $\pi(s)$ and state-based rewards $\mathcal{R}_s$

$$v_\pi(s) \leftarrow\!\shortmid\ s \bigcirc$$

$$v_\pi(s) \leftarrow \mathcal{R}_s$$

Bellman Expectation Equation for v_π (2)

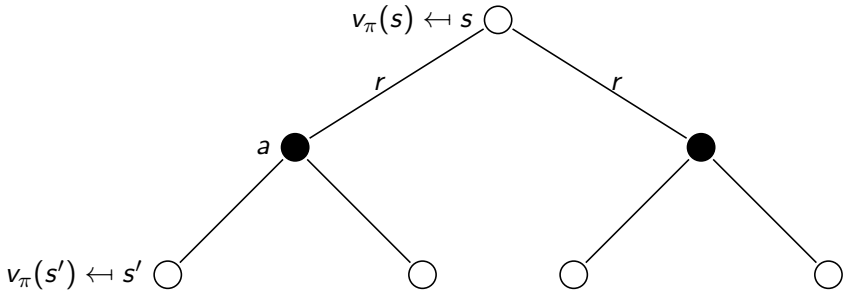
For deterministic policies $\pi(s)$ and state-based rewards $\mathcal{R}_s$



$$v_\pi(s) \leftarrow \mathcal{R}_s + [\mathcal{P}^a_{ss'}]$$

Bellman Expectation Equation for v_π (2)

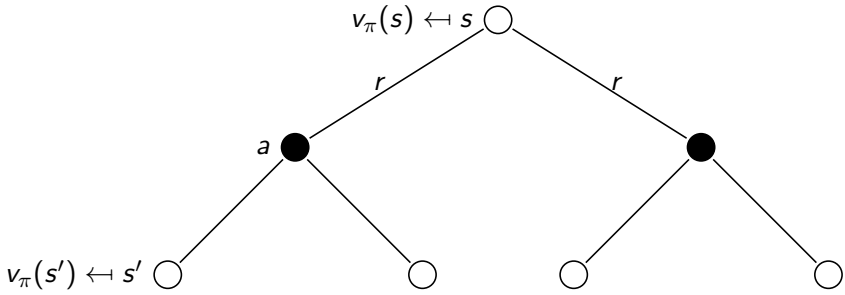
For deterministic policies $\pi(s)$ and state-based rewards $\mathcal{R}_s$



$$v_\pi(s) \leftarrow \mathcal{R}_s + \left[\sum_{s'} \mathcal{P}^a_{ss'} \, v_\pi(s') \right]$$

Bellman Expectation Equation for v_π (2)

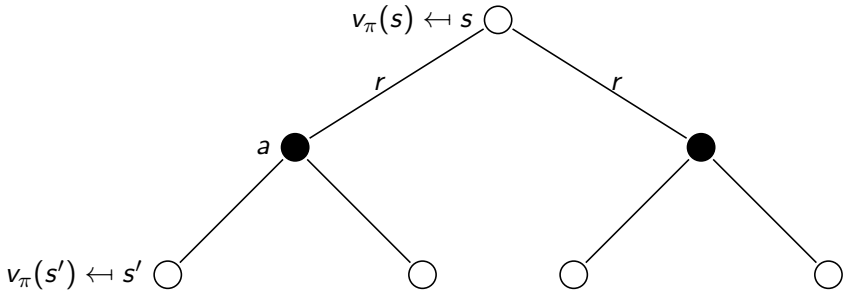
For deterministic policies $\pi(s)$ and state-based rewards $\mathcal{R}_s$



$$v_\pi(s) \leftarrow \mathcal{R}_s + \left[\gamma \sum_{s'} \mathcal{P}^a_{ss'} \, v_\pi(s') \right]$$

Bellman Expectation Equation for v_π (2)

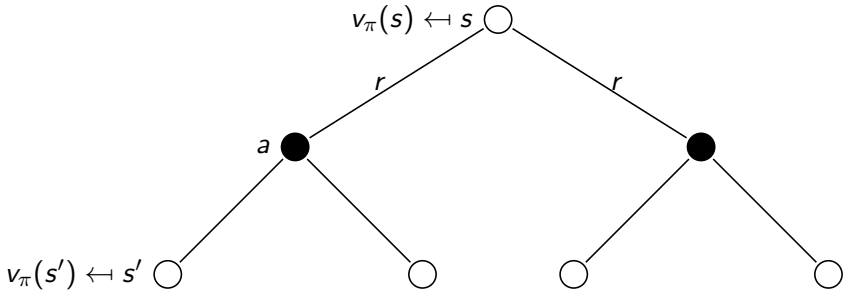
For deterministic policies $\pi(s)$ and state-based rewards $\mathcal{R}_s$



$$v_\pi(s) \leftarrow \mathcal{R}_s + \left[\gamma \sum_{s'} \mathcal{P}_{ss'}^{\pi(s)} \, v_\pi(s') \right]$$

Bellman Expectation Equation for v_π (2)

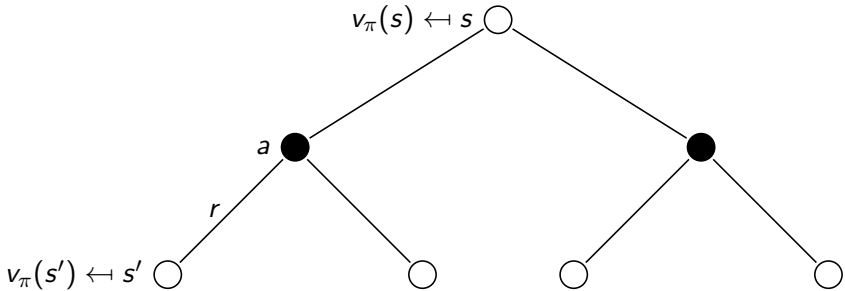
For deterministic policies $\pi(s)$ and state-based rewards $\mathcal{R}_s$



$$v_\pi(s) \leftarrow \begin{cases} \mathcal{R}_s \;\; \textit{if terminal} \\ \mathcal{R}_s + \left[\max_a \gamma \sum_{s'} \mathcal{P}_{ss'}^{\pi(s)} \, v_\pi(s')\right] \end{cases}$$

Back up function

Bellman Expectation Equation for v_π (3)



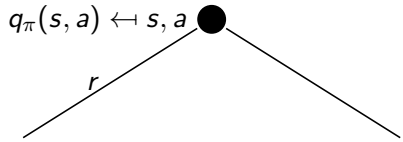
$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \, v_\pi(s') \right)$$

Bellman Expectation Equation for q_π (2)

$q_\pi(s, a) \leftarrow\!\shortmid\, s, a$ 

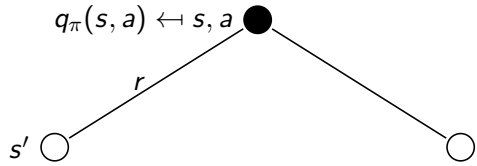
$q_\pi(s, a) = ?$

Bellman Expectation Equation for q_π (2)



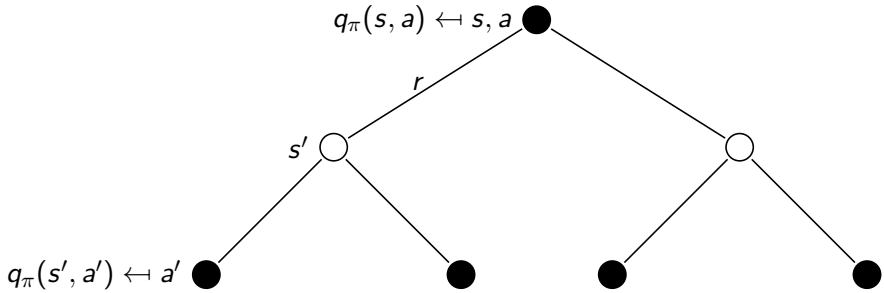
$$q_\pi(s, a) = \mathcal{R}_s^a$$

Bellman Expectation Equation for q_π (2)



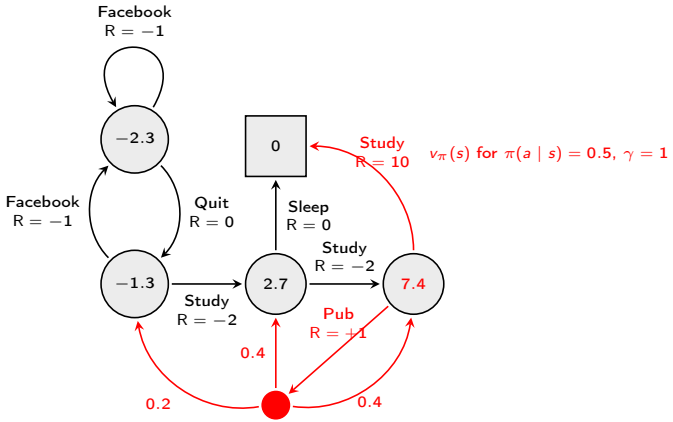
$$q_\pi(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a$$

Bellman Expectation Equation for q_π (2)

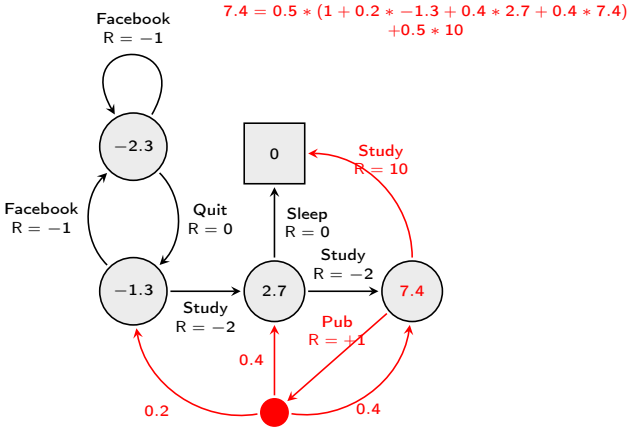


$$q_\pi(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a' \mid s') q_\pi(s', a')$$

Example: Bellman Expectation Equation in Student MDP



Example: Bellman Expectation Equation in Student MDP



Bellman Expectation Equation (Matrix Form)

The Bellman expectation equation can be expressed concisely using the induced MRP,

$$v_\pi = \mathcal{R}^\pi + \gamma \mathcal{P}^\pi v_\pi$$

with direct solution

$$v_\pi = (\mathrm{I} - \gamma \mathcal{P}^\pi)^{-1} \mathcal{R}^\pi$$

Optimal Value Function

Definition

The optimal state-value function $v_*(s)$ is the maximum value function over all policies

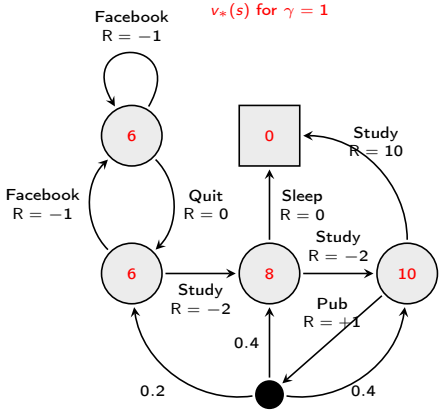
$$v_*(s) = \max_\pi v_\pi(s)$$

The optimal action-value function $q_*(s, a)$ is the maximum action-value function over all policies

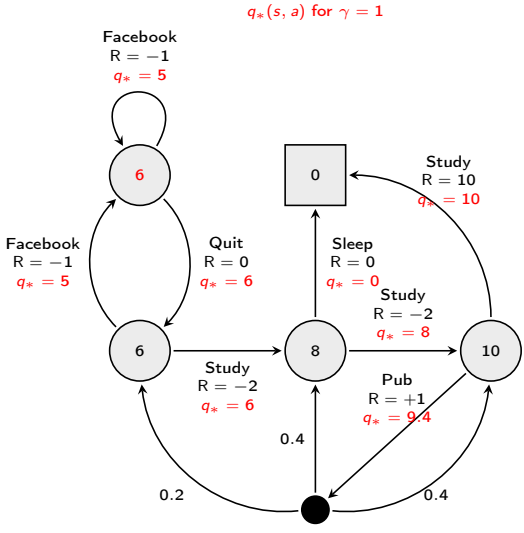
$$q_*(s, a) = \max_\pi q_\pi(s, a)$$

- The optimal value function specifies the best possible performance in the MDP.
- An MDP is "solved" when we know the optimal value fn.

Example: Optimal Value Function for Student MDP



Example: Optimal Action-Value Function for Student MDP



Define a partial ordering over policies

$$\pi \geq \pi' \text{ if } v_\pi(s) \geq v_{\pi'}(s), \forall s \in \mathcal{S}$$

Theorem

For any Markov Decision Process

- *There exists an optimal policy* π_* *that is better than or equal to all other policies,* $\pi_* \geq \pi, \forall \pi$
- *All optimal policies achieve the optimal value function,* $v_{\pi_*}(s) = v_*(s)$
- *All optimal policies achieve the optimal action-value function,* $q_{\pi_*}(s, a) = q_*(s, a)$

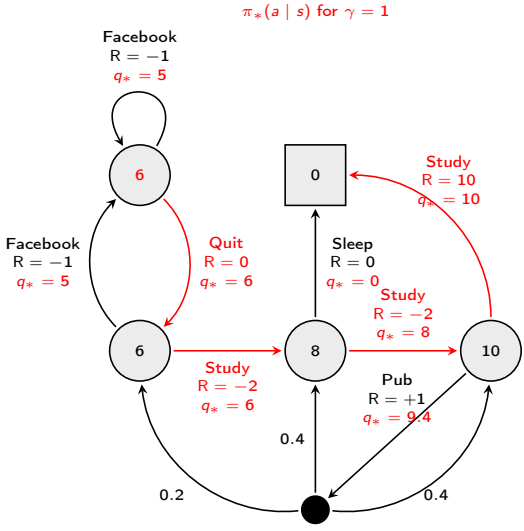
Finding an Optimal Policy

An optimal policy can be found by maximising over $q_*(s, a)$,

$$\pi_*(a \mid s) = \begin{cases} 1 & \text{if } a = \arg\max_{a \in \mathcal{A}} q_*(s, a) \\ 0 & \text{otherwise} \end{cases}$$

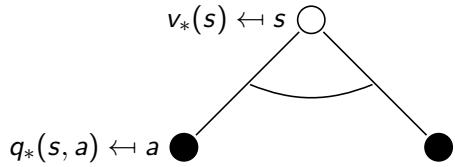
- There is always a deterministic optimal policy for any MDP
- If we know $q_*(s, a)$, we immediately have the optimal policy

Example: Optimal Policy for Student MDP



Bellman Optimality Equation for v_*

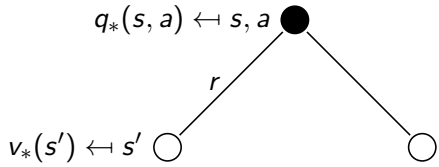
The optimal value functions are recursively related by the Bellman optimality equations:



$$v_*(s) = \max_a q_*(s,a)$$

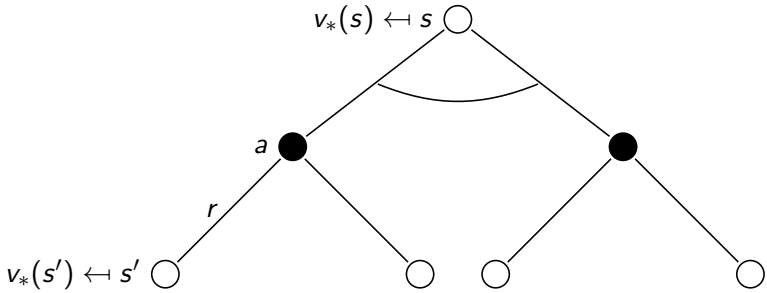
Bellman Optimality Equation for Q^*

The optimal value functions are recursively related by the Bellman optimality equations:



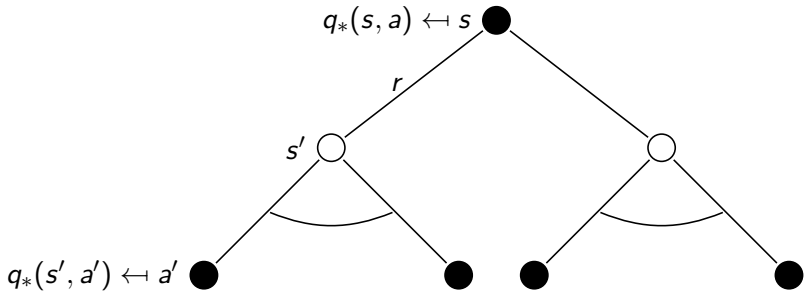
$$q_*(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \, v_*(s')$$

Bellman Optimality Equation for V^*



$$v_*(s) = \max_a \left[\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \, v_*(s') \right]$$

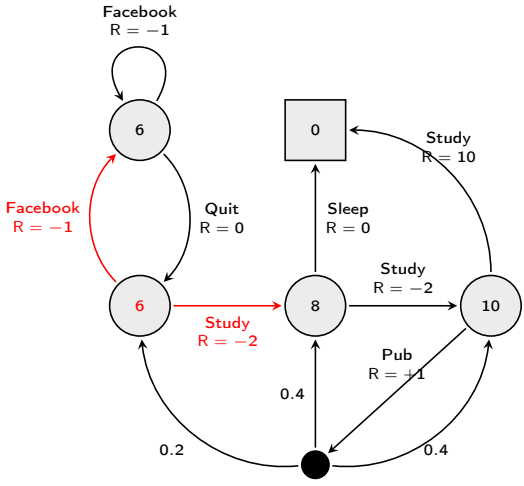
Bellman Optimality Equation for Q^*



$$q_*(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \max_{a'} q_*(s', a')$$

Example: Bellman Optimality Equation in Student MDP

$6 = \max(-2 + 8, -1 + 6)$



Solving the Bellman Optimality Equation

- Bellman Optimality Equation is non-linear
- No closed form solution (in general)
- Many iterative solution methods
  - Value Iteration
  - Policy Iteration
- Machine Learning methods for **approximating** the Bellman Equation
  - Q-learning
  - Sarsa

Outline

Extensions to MDPs

- Infinite and continuous MDPs
- Partially observable MDPs
- Undiscounted, average reward MDPs

Infinite MDPs

The following extensions are all possible:

- Countably infinite state and/or action spaces
  - Straightforward
- Continuous state and/or action spaces
  - Closed form for linear quadratic model (LQR)
- Continuous time
  - Requires partial differential equations
  - Hamilton-Jacobi-Bellman (HJB) equation
  - Limiting case of Bellman equation as time-step $\rightarrow 0$

A Partially Observable Markov Decision Process is an MDP with hidden states. It is a hidden Markov model with actions.

Definition

A POMDP is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{P}, \mathcal{R}, \mathcal{Z}, \gamma \rangle$

- $\mathcal{S}$ is a finite set of states
- $\mathcal{A}$ is a finite set of actions
- $\mathcal{O}$ is a finite set of observations
- $\mathcal{P}$ is a state transition probability matrix, $\mathcal{P}^a_{ss'} = \mathbb{P}\left[S_{t+1} = s' \mid S_t = s, A_t = a\right]$
- $\mathcal{R}$ is a reward function $\mathcal{R}^a_s = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$
- $\mathcal{Z}$ is an observation function, $\mathcal{Z}^a_{s'o} = \mathbb{P}\left[O_{t+1} = o \mid S_{t+1} = s', A_t = a\right]$
- A discount factor $\gamma \in [0, 1]$

POMDPs

- Current state is only **partially observable** — Partially Observable MDPs
- Planning, thus, is not done on the state of spaces
- Rather, planning is done over **belief states** — Probability distributions
- Policy then, is no longer a simple vector with a node for each state, but a multidimensional matrix with as many dimensions as there are states

Definition

A **history** H_t is a sequence of actions, observations and rewards,

$$\mathrm{H}_t = A_0, O_1, R_1, \ldots, A_{t-1}, O_t, R_t$$

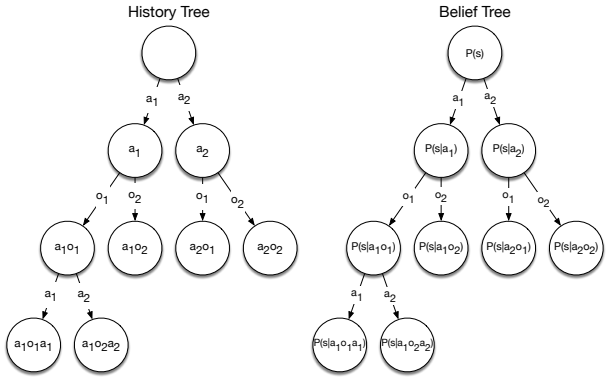
Definition

A belief state $b(h)$ is a probability distribution over states conditioned on the history h

$$b(h) = \left(\mathbb{P}\left[S_t = s^1 \mid H_t = h\right], \ldots \mathbb{P}\left[S_t = s^n \mid H_t = h\right]\right)$$

Reductions of POMDPs

- The history H_t satisfies the Markov Property
- The belief state $b(H_t)$ satisfies the Markov Property



- A POMDP can be reduced to an (infinite) history tree
- A POMDP can be reduced to an (infinite) belief state tree

Ergodic Markov Process

An ergodic Markov process is

- **Recurrent**: each state is visited an infinite number of times
- **Aperiodic**: each state is visited without any systematic period

Theorem

An ergodic Markov process has a limiting stationary distribution $\mu_\pi(\mathsf{s})$ *with the property*

$$\mu_\pi(\mathsf{s}) = \sum_{\mathsf{s}' \in \mathcal{S}} \mu_\pi(\mathsf{s}')\, \mathcal{P}_{\mathsf{s}'\,\mathsf{s}}$$

Definition

An MDP is ergodic if the Markov chain induced by any policy is ergodic.

For any policy π, an ergodic MDP has an average reward per time-step $\bar{\mathrm{R}}^{\pi}$ that is independent of start state.

$$\bar{\mathrm{R}}^{\pi} = \lim_{T \to \infty} \frac{1}{T} \mathbb{E}\left[\sum_{t=1}^{T} \mathrm{R}_t\right]$$

Average Reward Value Function

- The value function of an undiscounted, ergodic MDP can be expressed in terms of average reward.

Lecture Summary

- Stochastic Planning
- Utilities and the Expectation Operator
- Markov Processes
- Markov Reward Processes
- Markov Decision Processes
  - Formalism
  - Policies
- Extensions to MDP